Projeto em Dupla — CRUD em JavaScript

Oct 21, 2026 · @Tarik Ponciano

Sobre o projeto

Neste projeto, você e sua dupla vão construir um sistema em JavaScript que executa no terminal e gerencia os dados de um negócio. O objetivo é reunir tudo o que praticamos em sala em um programa completo: arrays, dicionários, laços, condicionais e entrada de dados com Inquirer.

O que é um CRUD? É a sigla para as quatro operações básicas de qualquer sistema que guarda dados:

Letra	Operação	No seu sistema
C	Create (criar)	Cadastrar
R	Read (ler)	Ver todos e ver um
U	Update (atualizar)	Editar
D	Delete (apagar)	Remover

Como vai funcionar

1. As duplas serão formadas por sorteio.
2. Cada dupla sorteia um número da lista de temas abaixo.
3. A dupla escolhe um **elemento** que faça parte do tema e define como os dados desse elemento serão guardados.
4. A dupla implementa o CRUD desse elemento com um menu de interação.
5. Ao final, a dupla entrega o link do repositório do projeto no GitHub.

Lista de temas

Cada dupla sorteia um número. O tema é só o ponto de partida: a escolha do elemento é da dupla.

Nº	Tema	Contexto
1	Clínica médica	Atendimentos, consultas e profissionais de saúde
2	Biblioteca	Acervo, empréstimos e leitores

Nº	Tema	Contexto
3	Academia	Treinos, alunos matriculados e equipamentos
4	Pet shop	Animais atendidos, serviços de banho e tosa, produtos
5	Locadora de veículos	Frota, aluguéis e clientes
6	Restaurante	Cardápio, pedidos e mesas
7	Hotel	Quartos, hóspedes e reservas
8	Oficina mecânica	Veículos em manutenção, serviços e peças
9	Cinema	Filmes em cartaz, sessões e salas
10	Loja de games	Jogos, consoles e acessórios
11	Farmácia	Medicamentos, estoque e receitas
12	Imobiliária	Imóveis para venda ou aluguel, visitas e proprietários
13	Agência de viagens	Pacotes, destinos e passageiros
14	Campeonato de futebol	Times, jogadores e partidas
15	Estacionamento	Vagas, veículos e mensalistas
16	Salão de beleza	Serviços, agendamentos e profissionais
17	Festival de música	Atrações, palcos e ingressos
18	Plataforma de streaming	Séries, filmes e episódios
19	Hortifrúti	Frutas e verduras, fornecedores e preços
20	ONG de adoção de animais	Animais para adoção, adotantes e voluntários

Etapa 1 — Elemento e estrutura de dados

Antes de programar, a dupla precisa decidir **o que** o sistema vai gerenciar e **como** os dados serão guardados.

1. Escolha o elemento

Escolha uma "coisa" do tema que precise ser cadastrada várias vezes. Exemplo: no tema **Clínica médica**, a dupla pode fazer um CRUD de **Paciente**.

2. Defina a estrutura de dados

Cada elemento será um dicionário, e todos os elementos ficarão em uma lista. Em outras palavras, uma lista de dicionários, como no carrinho de compras.

```
const pacientes = [
  { id: 1, nome: 'Maria Souza', idade: 34, telefone: '85 99999-1111',
convenio: 'Unimed' },
  { id: 2, nome: 'João Lima', idade: 52, telefone: '85 98888-2222', convenio:
'Particular' }
];
```

Regras da estrutura

1. O dicionário precisa ter **pelo menos 5 propriedades**.
2. Uma delas deve ser o `id`, um número que identifica cada elemento e é gerado automaticamente pelo sistema.
3. Pelo menos uma propriedade deve ser numérica, além do `id`, e pelo menos uma deve ser texto.
4. Os nomes das chaves devem ser claros e sem acentos ou espaços: `dataNascimento`, não `data de nascimento`.
5. O sistema deve começar com **3 elementos já cadastrados** no código, para facilitar os testes.

Antes de programar, mostre ao instrutor: o tema, o elemento escolhido e o dicionário com as propriedades e os tipos de cada uma.

Etapa 2 — Menu e operações

O programa exibe um menu que se repete até o usuário escolher sair. Troque "Paciente" pelo elemento da sua dupla.

```
===== CLÍNICA VIDA SAUDÁVEL =====
1 - Cadastrar paciente
2 - Ver todos os pacientes
3 - Ver um paciente
4 - Editar paciente
5 - Remover paciente
0 - Sair
```

1 — Cadastrar

- Peça cada dado do elemento, exceto o `id`.
- Gere o `id` automaticamente: o primeiro cadastrado depois dos 3 iniciais recebe 4, o seguinte recebe 5 e assim por diante.

- Adicione o novo dicionário à lista e exiba uma mensagem de sucesso com o `id` gerado.

2 — Ver todos

- Exiba todos os elementos de forma organizada, um por linha ou em blocos.
- Mostre também o total de elementos cadastrados.
- Se a lista estiver vazia, exiba uma mensagem avisando.

3 — Ver um

- Peça o `id` e exiba todos os dados daquele elemento.
- Se o `id` não existir, exiba `Paciente não encontrado`.

4 — Editar

- Peça o `id` do elemento a ser editado.
- Mostre os dados atuais e peça os novos valores de cada propriedade, exceto o `id`, que nunca muda.
- Atualize o dicionário na lista e exiba os dados atualizados.
- Se o `id` não existir, exiba a mensagem de não encontrado.

5 — Remover

- Peça o `id` do elemento a ser removido.
- Mostre os dados dele e peça confirmação: `Deseja remover? (s/n)`.
- Só remova da lista se a resposta for `s`.
- Se o `id` não existir, exiba a mensagem de não encontrado.

0 — Sair

- Encerre o programa com uma mensagem de despedida.
- Qualquer outra opção deve exibir `Opção inválida`.

Requisitos técnicos e dicas

Requisitos

- Projeto Node.js com `@inquirer/prompts`, usando `input()` para textos e `number()` para números.
- Os dados ficam em uma lista de dicionários, guardada em memória enquanto o programa estiver aberto.
- Use os recursos trabalhados em sala: variáveis, condicionais, laços, arrays e dicionários.
- Todo o código deve ser escrito pela dupla, e os dois integrantes devem saber explicar qualquer parte dele.

Dicas

- **Encontrar pelo id:** percorra a lista com `for` e compare `lista[i].id` com o `id` digitado. Guarde a posição `i` encontrada; ela será usada para ver, editar e remover. Se o laço terminar sem encontrar, o elemento não existe.
- **Gerar o id:** crie uma variável `proximoId` que começa em 4 e aumenta em 1 a cada cadastro.
- **Remover da lista:** use `lista.splice(posicao, 1)`, que retira 1 elemento a partir daquela posição.
- **Editar:** depois de encontrar a posição, altere as propriedades diretamente: `lista[posicao].nome = novoNome`.
- **Comece pelo mais simples:** faça o menu e o "Ver todos" primeiro, depois Cadastrar e Ver um. Editar e Remover reaproveitam a busca pelo id.
- **Teste os casos de erro:** lista vazia, id que não existe e opção inválida no menu.

Desafios extras (opcionais)

- Validar os dados no cadastro e na edição, como idade maior que zero ou nome não vazio.
- Na edição, permitir que o usuário deixe um campo em branco para manter o valor atual.
- Adicionar uma opção de busca por outro campo, como nome ou categoria.
- Adicionar uma opção de relatório com algum cálculo sobre os dados, como média, total ou maior valor.

Entrega

A entrega é o **link do repositório do projeto no GitHub**.

O repositório deve conter

- O arquivo do programa (`crud.js`).
- O `package.json` do projeto.
- Um `README.md` com o nome dos dois integrantes, o tema sorteado, o elemento escolhido e como executar o projeto (`npm install` e depois `node crud.js`).
- Um arquivo `.gitignore` com a linha `node_modules`, para que essa pasta não seja enviada.

Antes de enviar o link, confira se o repositório está público e abra o link em uma janela anônima para garantir que ele funciona.